Credit Card Fraud Detection using Random Forest (KNIME Workflow)

This project implements an end-to-end **Credit Card Fraud Detection System** using **KNIME Analytics Platform** and a **Random Forest classifier**.
The objective is to automatically identify potentially fraudulent credit card transactions based on transaction behavior patterns and probability thresholds.

The workflow is designed to support:

- Model training and evaluation
- Model persistence for deployment
- Scoring of new/unseen transaction data
- Aggregation and visualization of fraud detection results

The workflow follows industry-aligned fraud analytics practices and is suitable for batch processing, monitoring, and deployment in real-world finance environments.

Index / Table of Contents

Sr. No.	Section Title	Description
1	**Project Overview**	Project objective and workflow goals
2	**Dataset Description**	Dataset background, features, and target variable
3	**Workflow Architecture (High-Level)**	Logical stages of the end-to-end KNIME workflow
4	**Step-by-Step Workflow Explanation**	Detailed explanation of training workflow components
5	**Step-by-Step Workflow Explanation**	Training data ingestion, Random Forest model training, evaluation, and model persistence
6	**Reading Model and New Data (Deployment Phase)**	Loading trained model and unseen transaction data
7	**Fraud Prediction on New Data**	Applying model and probability-based fraud classification
8	**Result Aggregation and Visualization**	Fraud count aggregation and visualization
9	**Fraud Detection Logic**	Probability thresholding and business logic

Sr. No.	Section Title	Description
10	**Key Advantages of the Approach**	Strengths of the Random Forest–based solution
11	**Conclusion**	Summary and future enhancement possibilities
12	**Tools & Technologies Used**	Platforms, models, and libraries used

1. Project Overview

Objective

Build and deploy a machine learning model in KNIME to classify transactions as **fraudulent** or **legitimate** using a **Random Forest** classifier.

Workflow Goals

- Train and evaluate a supervised learning model.
- Save and reload the model for deployment.
- Predict fraud probability for new data batches.
- Generate interpretable visualization summaries.

---

2. Dataset Description

We use a popular dataset available from Kaggle called Credit Card Fraud Detection [https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud]. This dataset is composed of real, anonymized transactions made by credit cards in September 2013 by European cardholders. It includes 284,807 transactions over two days, containing 492 fraudulent transactions. The dataset represents a severe class imbalance between the 'no fraud' (0) and 'fraud' (1), where 'frauds' account for only 0.172% of the data.

2.1 Features

The dataset follows the standard credit card fraud dataset schema:

- **Time** → Time elapsed since the first transaction
- **V1 – V28** → PCA-transformed numeric components (anonymized)
- **Amount** → Transaction amount

2.2 Target Variable

- **Class**
  - `1` → Fraudulent transaction

- `0` → Legitimate transaction

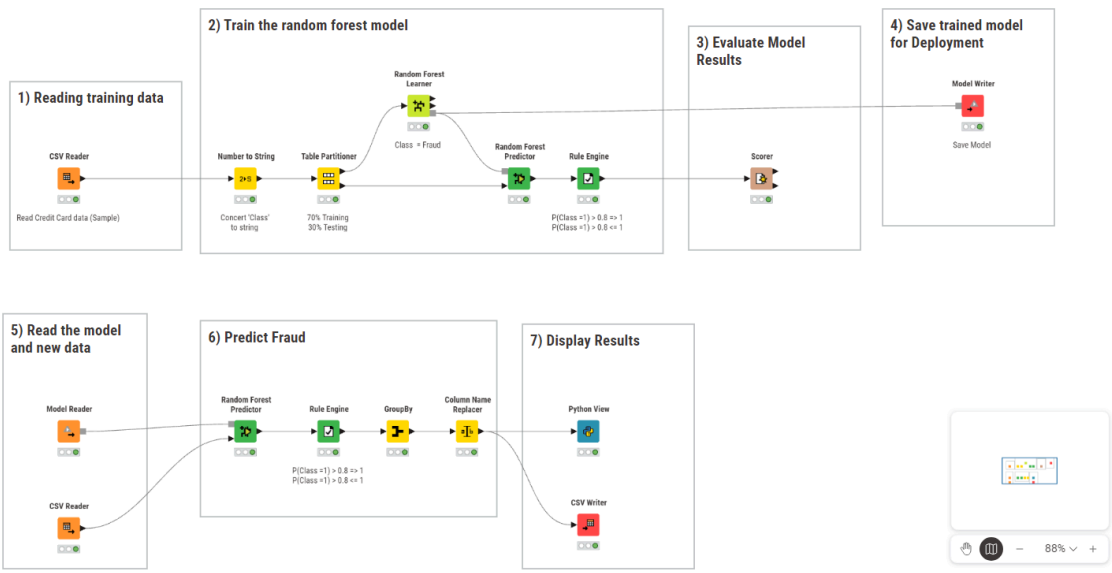
---

3. Workflow Architecture (High-Level)

The complete KNIME workflow is divided into **seven logical stages**:

1. Reading training data
2. Training the Random Forest model
3. Evaluating model performance
4. Saving the trained model
5. Loading model and new transaction data
6. Predicting fraud on new data
7. Aggregating and displaying results

Each stage is implemented using dedicated KNIME nodes for modularity, reproducibility, and easy maintenance.



---

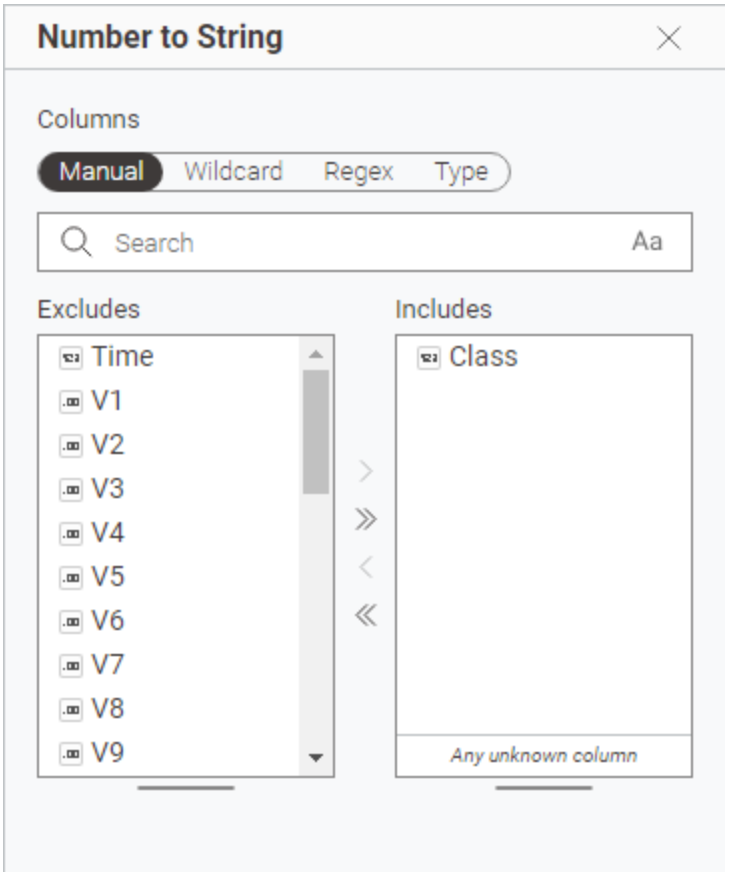
4. Step-by-Step Workflow Explanation

4.1 Reading Training Data

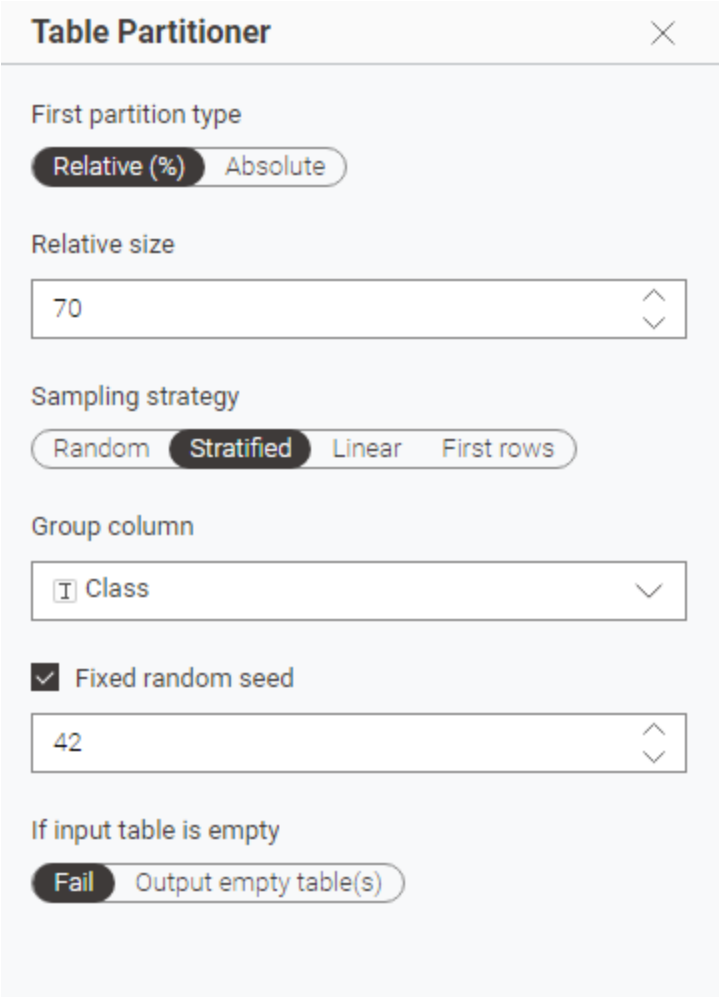
- **CSV Reader**
  - Loads historical credit card transaction data.
  - Ensures all feature columns are numeric.
  - Retains `Class` column for supervised learning.

4.2 Model Training (Random Forest)

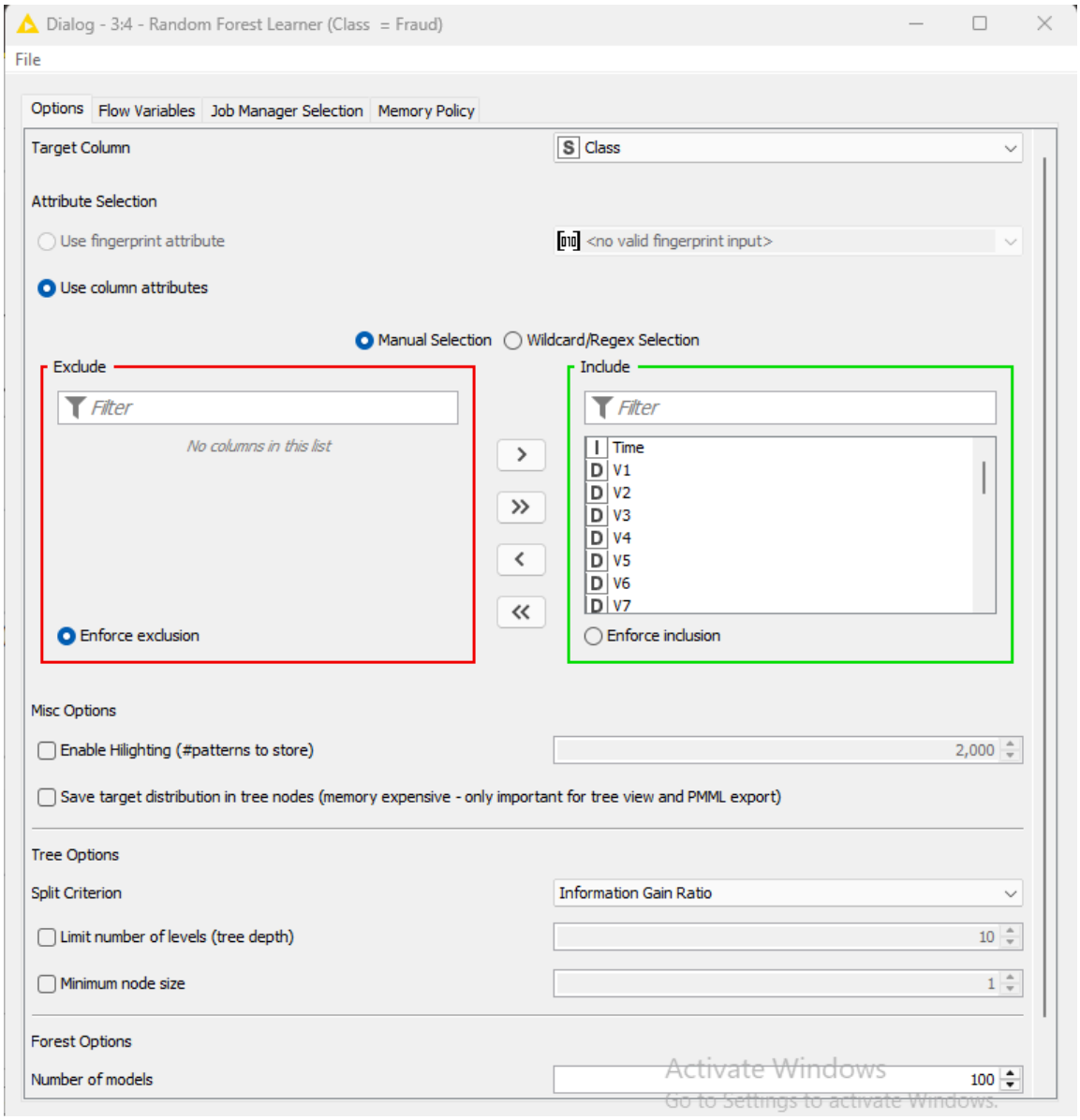
- **Number to String** → Converts `Class` into nominal type for KNIME.



- **Table Partitioning** → Splits data into 70% Training and 30% Testing sets.

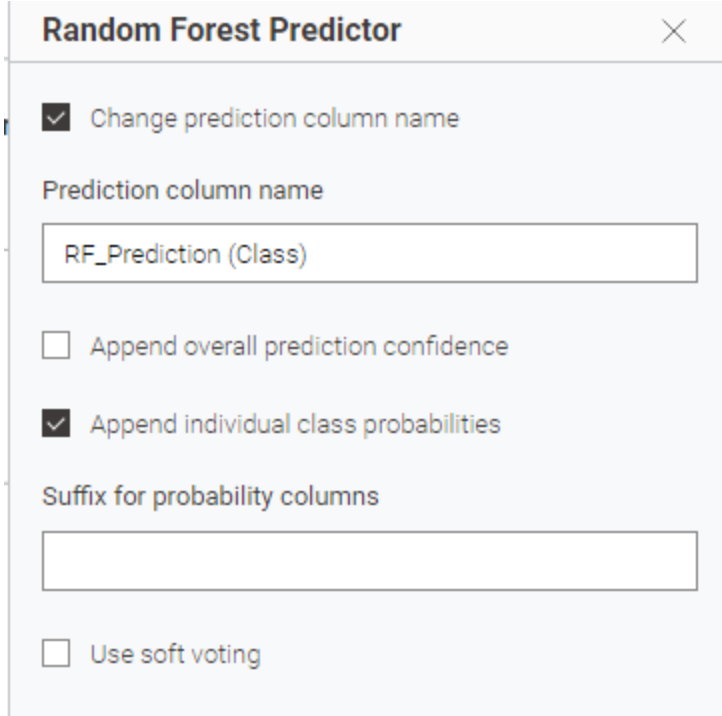


- **Random Forest Learner**
  - Target variable: `Class`
  - Ensemble-based classification with multiple trees
  - Handles non-linear patterns and noise efficiently

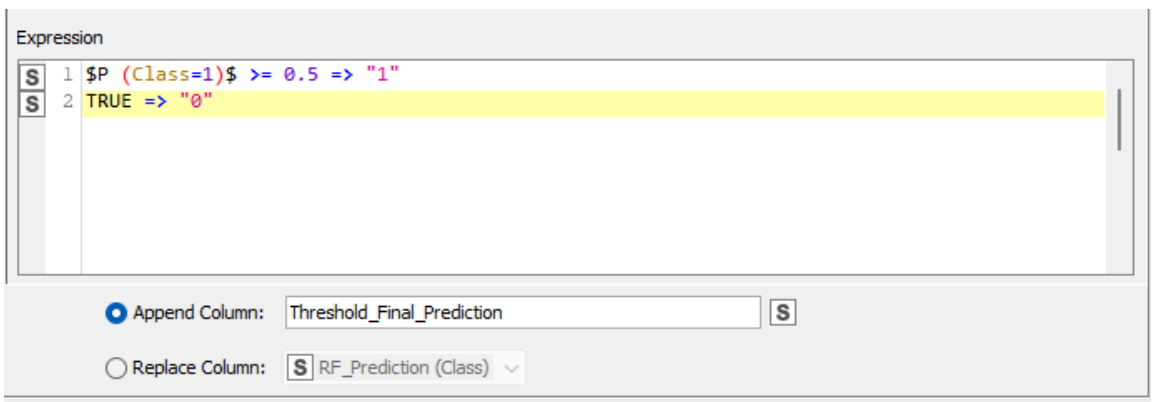


4.3 Model Evaluation

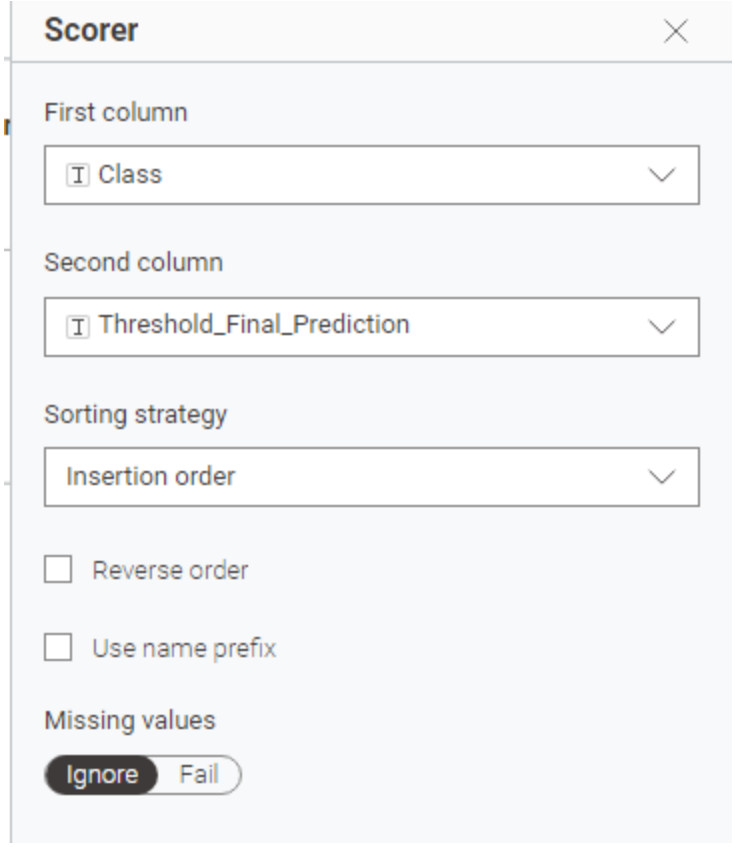
- **Random Forest Predictor** → Generates predictions on test dataset.
- Outputs:
  - Predicted class
  - Probability scores (`P(Class=0)` , `P(Class=1)`)



- **Rule Engine** → Applies fraud decision threshold:
  - If `P(Class=1) > 0.8` → mark as **Fraud**



- **Scorer** → Computes performance metrics:
  - Accuracy, Precision, Recall, F1-score
  - Confusion Matrix



4.4 Saving the Trained Model

- **Model Writer**
  - Persists trained Random Forest model on disk.
  - Allows reuse for future inference workflows.

---

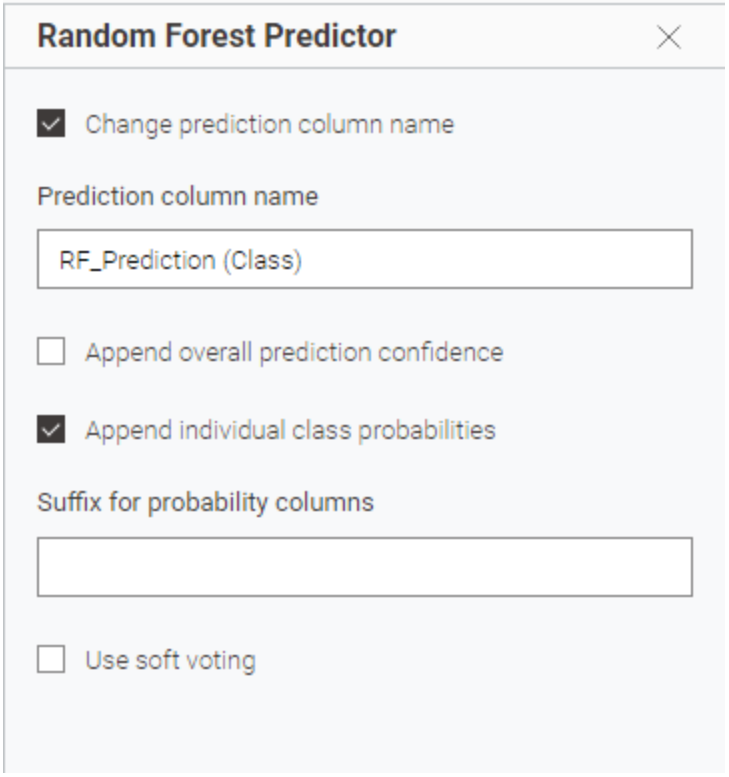
5 Reading Model and New Data (Deployment Phase)

- **Model Reader** → Loads previously saved Random Forest model.
- **CSV Reader** → Loads new unseen transaction batch (without target `Class`).

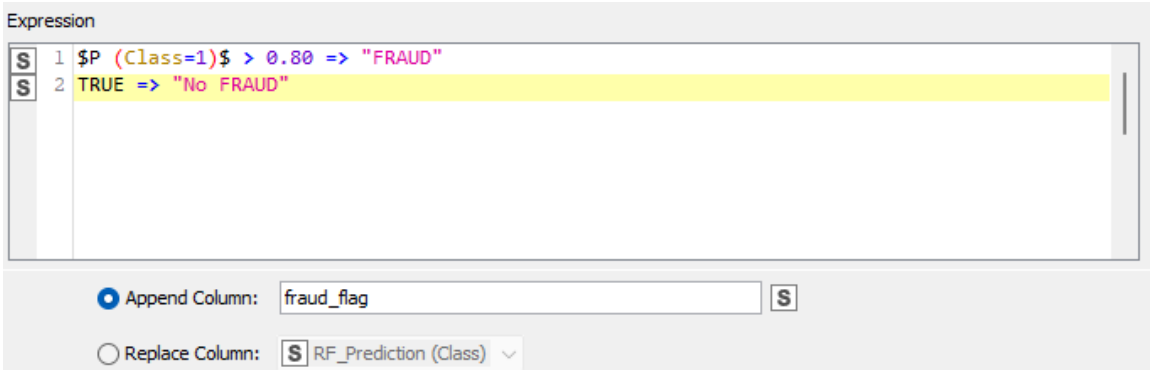
---

6 Fraud Prediction on New Data

- **Random Forest Predictor** → Generates fraud probability for new transactions.



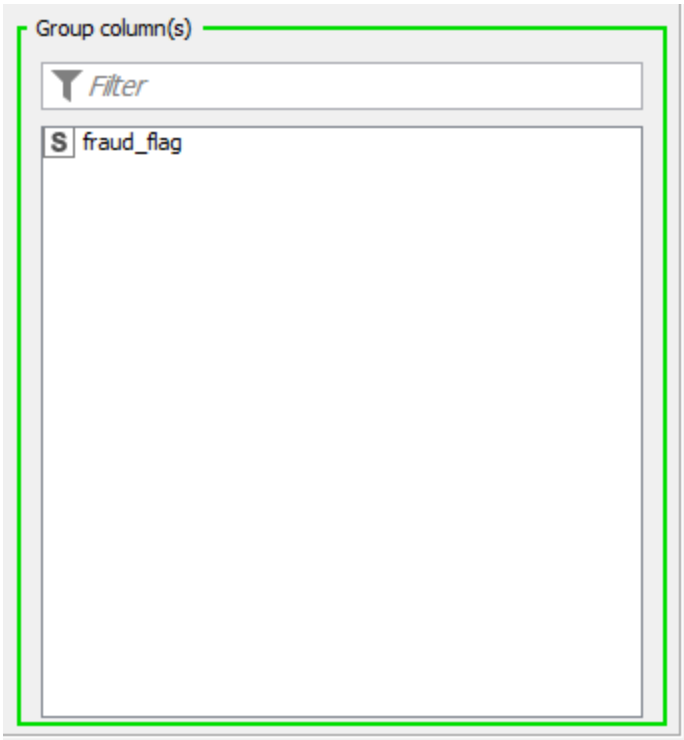
- **Rule Engine** → Applies same thresholding logic:
  - `P(Class=1) > 0.8 → FRAUD`
  - Else → NON-FRAUD

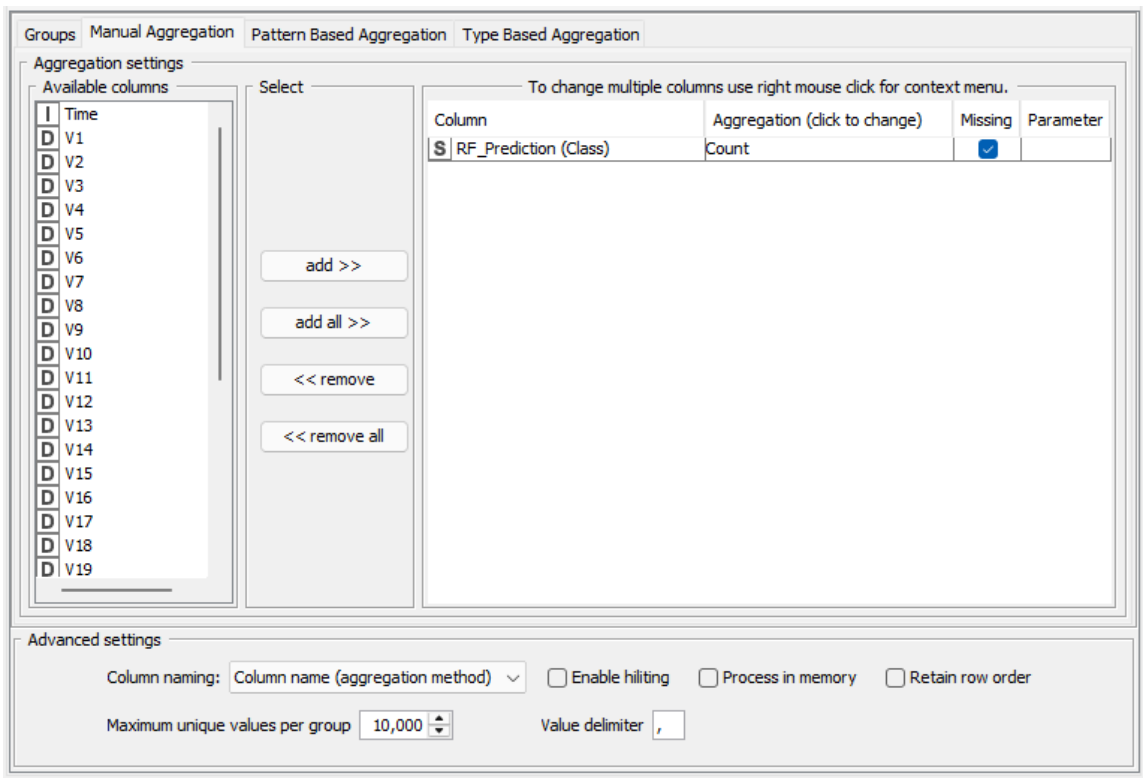


---

7 Result Aggregation and Visualization

- **GroupBy** → Groups by fraud flag and manual aggregation of number of fraud and non-fraud cases using `Count`.





- **Column Name Replacer** → Replace the Column name `RF_Prediction (Class) (Count)` with `Count`.
- **Python Script (View)** → Visualizes results (fraud vs non-fraud counts) using matplotlib.

```
# Placeholder for KNIME Python View Node
# Example visualization of Fraud vs Non-Fraud transactions
import knime.scripting.io as knio
import matplotlib.pyplot as plt
```

```python
# Read input table
df = knio.input_tables[0].to_pandas()

# RowID is the index in KNIME
categories = df["fraud_flag"]
Counts = df["Count"]

# Create bar chart
fig = plt.figure()
plt.bar(categories, Counts)
plt.xlabel("Transaction Type")
plt.ylabel("Count")
plt.title("Fraud vs No Fraud Count")
plt.yticks(range(0, int(Counts.max()) + 1))

# Show values on bars
for i, v in enumerate(Counts):
    plt.text(i, v, str(v), ha='center', va='bottom')

# Save as PNG (relative to KNIME workflow folder)
plt.savefig("../Output/fraud_vs_no_fraud.png", dpi=300,
bbox_inches="tight")

# Output to KNIME view
knio.output_view = knio.view(fig)
```

---

8. Fraud Detection Logic

System uses **probability-based detection** instead of fixed label classification.

Rule:
Flag transaction as fraud if:

`P(Class = 1) > 0.8`

Advantages:

- Reduces false positives
- Tunable sensitivity
- Aligns with real-world fraud risk systems

---

9. Key Advantages of the Approach

- Ensemble-based Random Forest ensures robustness
- Probability threshold allows business-tuned sensitivity
- KNIME workflow modular and reusable for production

- Clear separation of training, evaluation, and deployment
- Visualization provides transparent fraud distribution insight

---

10. Conclusion

This project demonstrates a complete, production-ready **Credit Card Fraud Detection pipeline** using KNIME.
It covers both **model development** and **deployment scoring**, providing a foundation for enterprise fraud monitoring.

Potential Extensions:

- Add ROC/PR curve analysis for threshold tuning
- Perform time-based fraud trend analysis
- Integrate with alerting or dashboard systems
- Automate via KNIME Server scheduling

---

11. Tools & Technologies Used

- KNIME Analytics Platform
- Random Forest Classifier
- Python (matplotlib for visualization)
- CSV-based batch data processing

🎉 End of Project